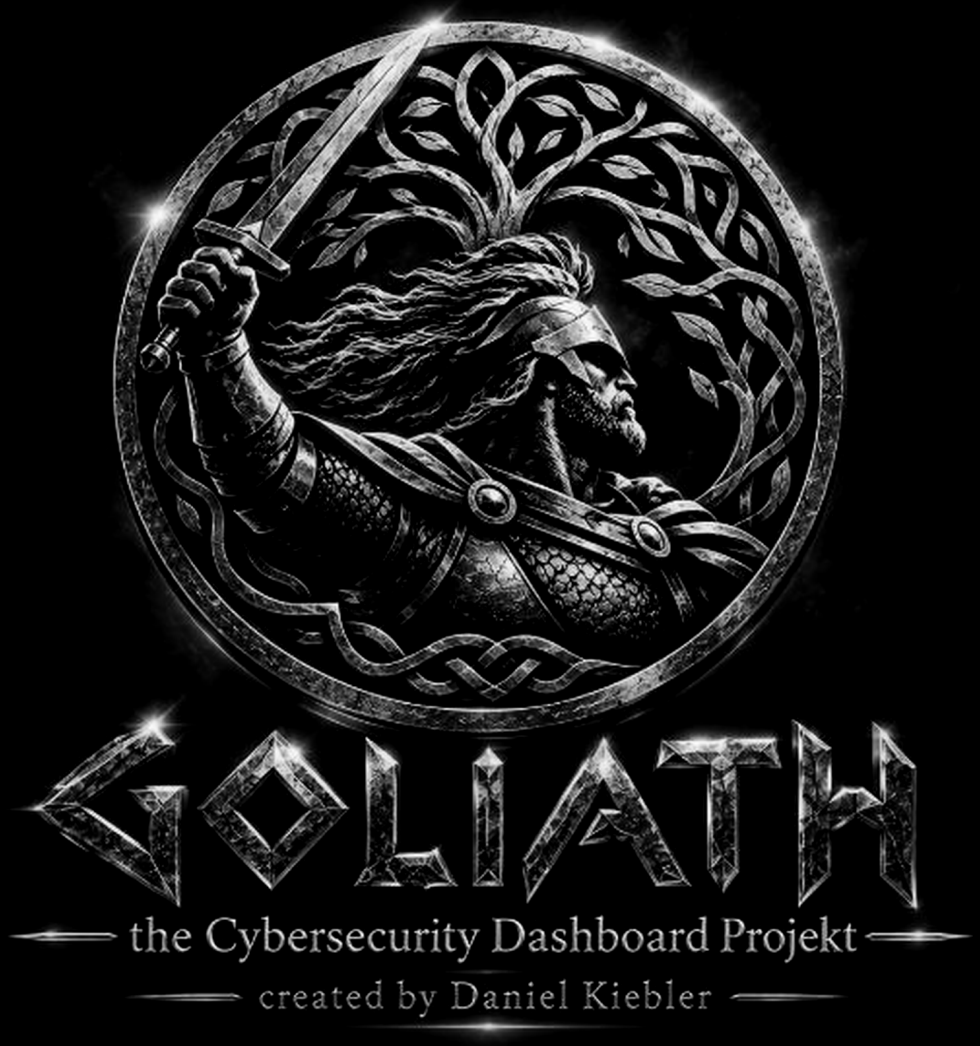


PROJEKTDOKUMENTATION

Windows-Dashboard 1.1 · Windows-Agent · Android-App 2.0.0

Dokumentversion 1.0 · Stand: 13. Oktober 2026

Inhaltsverzeichnis

1 Projektüberblick . . . 3
1.1 Ziele . . . 3
1.2 Bestandteile und Releasestände . . . 3
1.3 Abgrenzung . . . 3
2 Systemarchitektur und Rollenmodell . . . 4
2.1 Rollen . . . 4
2.2 Betriebsvarianten . . . 5
2.3 Identität und Datenhaltung der Agenten . . . 5
3 Funktionsumfang (aktueller Stand) . . . 5
4 Weboberfläche . . . 6
5 Windows-Agent . . . 6
5.1 Ablauf . . . 6
5.2 Erfasste Daten . . . 7
5.3 Datenschutz . . . 7
6 Kommunikation und Netzwerkaufbau . . . 7
6.1 Netzwerkparameter . . . 7
6.2 Ablauf eines Berichts . . . 8
6.3 Beispiel für einen Bericht . . . 8
6.4 Antwortcodes . . . 8
6.5 Anforderungen an das Netzwerk . . . 9
7 Dashboard-Server: Referenz . . . 9
7.1 Endpunkte . . . 9
7.2 Datenhaltung . . . 9
7.3 CVE/CVSS-Abgleich . . . 10
8 Android-App (Version 2.0.0) . . . 10
8.1 Komponenten . . . 10
8.2 Erreichbarkeit der Endpunkte . . . 11
8.3 Berechtigungen . . . 11
8.4 Android-17-Anpassungen . . . 11
8.5 Unterschiede zum Windows-Dashboard . . . 11
8.6 Projektstatus der App . . . 12
9 Technische Anforderungen . . . 12
10 Installation und Inbetriebnahme . . . 12
10.1 Windows-Master und Agent . . . 12
10.2 Android-Master . . . 13
11 Sicherheit und Datenschutz . . . 13
11.1 Umgesetzte Schutzmaßnahmen . . . 13
11.2 Bekannte Risiken und Empfehlungen . . . 13
12 Bekannte Grenzen . . . 14
13 Anhang . . . 14
13.1 Dateiübersicht . . . 14
13.2 Glossar . . . 15
13.3 Versionshistorie . . . 15

1 Projektüberblick

Goliath – the Cybersecurity Dashboard Projekt ist ein lokales Sicherheits-Dashboard für die autorisierte Inventarisierung von Windows-Rechnern im eigenen Netzwerk. Ein zentraler Dashboard-Server (**Master**) nimmt Inventarberichte von Windows-Agenten entgegen, stellt sie in einer Weboberfläche dar und gleicht die erfasste Software mit der öffentlichen CVE-Datenbank der NVD ab, inklusive CVSS-Bewertung.

1.1 Ziele

- Erfassen von installierter Software (Name, Version, Hersteller) und laufenden Prozessen (Name, PID, Pfad) je Rechner.
- Zentrale, lokale Darstellung aller gemeldeten Geräte mit Kennzahlen, Software- und Prozesslisten.
- Erster Abgleich von Softwarebezeichnung und Versionsstring mit CVE-Einträgen und CVSS-Werten.
- Einfacher Erreichbarkeitstest (Ping) für einzelne private IP-Adressen.
- Betrieb ohne Cloud-Dienst: Die Daten bleiben auf dem Master-Gerät.

1.2 Bestandteile und Releasestände

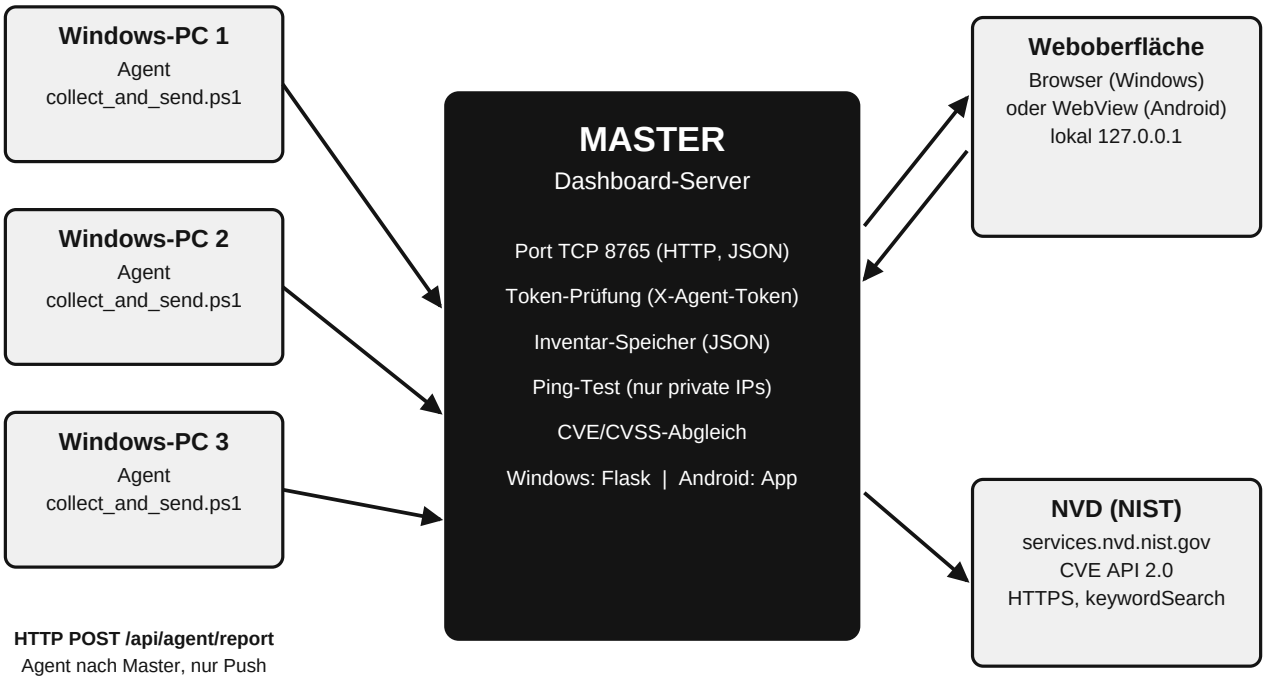
Bestandteil	Rolle	Stand
Dashboard-Server (Python/Flask)	Master auf einem Windows-PC: API, Speicherung, CVE-Abgleich, Auslieferung der Oberfläche	Version 1.1 (11.10.2026)
Windows-Agent (PowerShell)	Sammelt Inventar auf dem Ziel-PC und sendet es per HTTP an den Master	Teil von Version 1.1
Weboberfläche (HTML/CSS/JS)	Darstellung von Kennzahlen, Geräten, Software, Prozessen, CVE-Ergebnissen	Teil von Version 1.1
Android-App (Java)	Master auf dem Android-Handy: eingebetteter Server mit identischer Oberfläche	Version 2.0.0 (13.10.2026), Quellcode

1.3 Abgrenzung

Die Anwendung führt **keine** Netzwerk- oder Portscans, keine Exploits, keine Erfassung von Zugangsdaten und keine beliebigen Remote-Befehle aus. Das ESP32-S3-Board ist für die Dashboard- und Agent-Funktion nicht erforderlich; der Abschnitt „ESP32-S3 Verbindung“ der Oberfläche ist ein Hinweis, keine implementierte Funktion. Der Einsatz ist auf Systeme beschränkt, die man besitzt oder administrieren darf.

2 Systemarchitektur und Rollenmodell

Das System folgt einem einfachen Master-Agent-Modell. Der Master ist der einzige zentrale Punkt, die Agenten sind reine Datenlieferanten. Die Kommunikation verläuft ausschließlich vom Agenten zum Master.



2.1 Rollen

Rolle	Aufgaben	Initiiert Verbindungen?
Master (Dashboard-Server)	Nimmt Berichte entgegen, prüft Token und Inhalt, speichert das Inventar, liefert die Oberfläche, führt Ping und CVE-Abgleich aus.	Nur auf Benutzeraktion: Ping an eine private IP, HTTPS-Anfrage an die NVD.
Agent (Windows-PC)	Liest Rechnername, IPv4, Betriebssystem, installierte Software und Prozesse und sendet einen Bericht.	Ja, einmalig pro manuellem Start (HTTP POST an den Master).
Betrachter	Sieht die Oberfläche: Browser auf dem Master-PC (Windows) bzw. WebView in der App (Android).	Nein, nur lokal (127.0.0.1).

2.2 Betriebsvarianten

Variante	Master	Agenten	Erreichbarkeit
A Lokal (Standard)	Windows-PC, Bindung 127.0.0.1	Derselbe PC	Nur dieser PC
B LAN-Master (Windows)	Windows-PC, Umgebungsvariable GOLIATH_BIND=0.0.0.0	Weitere PCs im LAN	Gesamte Server-API im LAN erreichbar (siehe Abschnitt 11)
C Android-Master	Android-App, Bindung 0.0.0.0 auf Port 8765	Windows-PCs im selben WLAN	Aus dem LAN nur Agent-Upload und /health, Oberfläche nur in der App

2.3 Identität und Datenhaltung der Agenten

- Ein Agent ist durch die Kombination aus **host** und **ip** identifiziert. Ein neuer Bericht desselben Paares ersetzt den alten.
- Der Master speichert die letzten **100** Berichte.
- Der Master ruft Agenten nie ab (kein Pull) und sendet keine Befehle an Agenten.

3 Funktionsumfang (aktueller Stand)

Funktion	Beschreibung	Windows 1.1	Android 2.0.0
Agent-Empfang	POST /api/agent/report mit Token-Prüfung und Inhaltsvalidierung	Ja	Ja
Inventarspeicher	Persistente JSON-Datei, Ersetzung je host+ip, maximal 100 Berichte	Ja	Ja
Kennzahlen	Erfasste Hosts, als online gemeldet, Software-Einträge, Prozesse	Ja	Ja
Geräteliste	Host, IP, Betriebssystem, Anzahl Software/Prozesse, letzter Bericht	Ja	Ja
Software-Inventar	Programm, Version, Hersteller je Gerät (bis 1000 Zeilen angezeigt)	Ja	Ja
Prozessliste	Name, PID, Pfad mit Textfilter (bis 1000 Zeilen angezeigt)	Ja	Ja
IP-Diagnose (Ping)	Erreichbarkeit einer privaten, Loopback- oder Link-Local-Adresse	Ja (System-Ping)	Ja (isReachable)
CVE/CVSS-Abgleich	NVD-Schlüsselwortsuche je Software (bis 100 Einträge), Score und Schweregrad	Ja	Ja
Automatische Aktualisierung	Oberfläche lädt das Inventar alle 30 Sekunden neu	Ja	Ja
Token-Verwaltung	Windows: Umgebungsvariable bzw. Eingabe beim Start. Android: Dialog „Server“, Zufallstoken, Kopieren	Ja	Ja
Hintergrundbetrieb	Server läuft als Vordergrunddienst mit Benachrichtigung und Beenden-Aktion	Fenster offen lassen	Ja
ESP32-S3 / USB	Nur Hinweistext, keine Funktion	Nein	Nein

4 Weboberfläche

Die Oberfläche ist eine einzelne Seite im dunklen Design (Kopfzeile „Cybersecurity Dashboard“, Hero-Bereich, Kennzahlen-Kacheln, nummerierte Bereiche). Auf Android wird dieselbe Seite unverändert im WebView dargestellt; einzige Textanpassung: Die Statusanzeige lautet „Lokales Netzwerk“ statt „Nur localhost“.

Bereich	Inhalt
Kennzahlen	Vier Kacheln: Erfasste Hosts, Als online gemeldet (Stand des letzten Berichts), Softwareeinträge, Prozesse.
01 / Network – IP-Diagnose	Eingabe einer privaten IP, Ping-Ergebnis, Anzeige der lokalen Server-IP.
02 / Inventory – Agent-Inventar	Hinweis zum Agent-Start, Schaltfläche „Inventar aktualisieren“.
03 / Asset Inventory – Erfasste Geräte	Tabelle aller Hosts mit Schaltfläche „Ansehen“, die das Gerät für Software und Prozesse auswählt.
04 / Software – Software-Inventar	Geräteauswahl und Tabelle Programm / Version / Hersteller.
05 / Process Review – Aktive Prozesse	Filter nach Name, PID oder Pfad; Tabelle Prozess / PID / Pfad.
06 / Vulnerability Intelligence – CVE/CVSS	Geräteauswahl, Start des Abgleichs, Ergebnistabelle Software / CVE (Link zur NVD) / CVSS / Schweregrad / Beschreibung.
07 / USB Device – ESP32-S3	Erklärender Hinweis: Das Board ist kein Windows-Agent; keine versteckten USB-Befehle.

5 Windows-Agent

Der Agent besteht aus `COLLECT_AND_SEND.bat` (interaktiver Starter) und `collect_and_send.ps1` (Inventarisierung und Upload). Er wird **manuell** gestartet; es gibt keinen Dienst und keine geplante Aufgabe.

5.1 Ablauf

- Die .bat-Datei fragt Dashboard-URL und Agent-Token ab und ruft das PowerShell-Skript auf.
- Das Skript ermittelt Rechnername, die erste bevorzugte IPv4-Adresse (ohne 127.* und 169.254.*; Ersatzwert 127.0.0.1) und die Betriebssystembezeichnung.
- Es liest die installierte Software aus den Uninstall-Registry-Schlüsseln (HKLM, HKLM WOW6432Node, HKCU), nicht über Win32_Product.
- Es liest alle laufenden Prozesse (Name, PID, Pfad, soweit lesbar).
- Es sendet den Bericht per HTTP POST mit Header `X-Agent-Token` und gibt die Zahl der übertragenen Einträge aus.

5.2 Erfasste Daten

Feld	Inhalt	Grenze
host	Rechnername (COMPUTERNAME)	–
ip	Erste bevorzugte IPv4-Adresse	Muss privat, Loopback oder Link-Local sein
os	Betriebssystembezeichnung (Win32_OperatingSystem.Caption)	–
online	Immer true beim Senden	Kein Live-Status
collected_at	Zeitpunkt der Erfassung (UTC, ISO 8601)	–
software[]	name, version, publisher; sortiert, Duplikate entfernt	Maximal 2000 Einträge
processes[]	name, pid, path (Pfad kann leer sein)	Maximal 2000 Einträge

Hinweis zum Online-Status
„Als online gemeldet“ bedeutet nur, dass der letzte Bericht dieses Geräts eingegangen ist. Da Agenten nicht dauerhaft laufen, ist der Wert kein Live-Zustand. Für eine aktuelle Prüfung dient der Ping in Bereich 01.

5.3 Datenschutz

Der Agent sammelt Hostname, IPv4, Betriebssystem, Software und Prozesse. Vor dem Einsatz sind Zweck, Rechtsgrundlage, Transparenz, Aufbewahrung und Zugriffsrechte zu klären; ausgeführt werden darf er nur auf ausdrücklich berechtigten Geräten.

6 Kommunikation und Netzwerkaufbau

6.1 Netzwerkparameter

Parameter	Wert
Transport	TCP, HTTP/1.1 (unverschlüsselt)
Port des Masters	8765
Nutzdaten	JSON, UTF-8
Richtung	Agent an Master (HTTP POST); Master sendet nur die Antwort
Authentifizierung	Header X-Agent-Token, muss exakt dem Token des Masters entsprechen
Adressierung	Der Agent erhält die Master-URL manuell, z. B. http://192.168.1.40:8765 (keine automatische Erkennung)
Netz	Gleiches LAN bzw. WLAN; Quelle des Berichts muss eine private Adresse sein (Android-Master prüft die Quell-IP)

6.2 Ablauf eines Berichts

Schritt	Akteur	Aktion
1	Master	Server läuft und lauscht auf Port 8765; Token ist gesetzt.
2	Agent	Erfasst Inventar und baut das JSON-Dokument.
3	Agent	Sendet POST /api/agent/report mit X-Agent-Token.
4	Master	Prüft: Token gesetzt und korrekt (403/401), Pflichtfelder host und ip (400), gültige private IP (400).
5	Master	Fügt received_at (UTC) hinzu, ersetzt vorhandenen Bericht desselben host+ip, behält die letzten 100.
6	Master	Schreibt die Inventardatei und antwortet mit ok, host, software_items, process_items.
7	Betrachter	Oberfläche lädt /api/summary (beim Öffnen, per Knopf, alle 30 s) und zeigt die Daten an.

6.3 Beispiel für einen Bericht

```
POST /api/agent/report HTTP/1.1
Content-Type: application/json; charset=utf-8
X-Agent-Token: <Token>

{
  "host": "BUERO-PC01",
  "ip": "192.168.1.25",
  "os": "Microsoft Windows 11 Pro",
  "online": true,
  "collected_at": "2026-10-13T08:15:00.0000000Z",
  "software":  [ { "name": "7-Zip", "version": "24.08", "publisher": "Igor Pavlov" } ],
  "processes": [ { "name": "explorer", "pid": 4312, "path": "C:\\Windows\\explorer.exe" } ]
}
```

6.4 Antwortcodes

Code	Bedeutung
200	Bericht angenommen
400	Ungültiger Bericht, ungültige oder nicht private IP-Adresse
401	Ungültiger Agent-Token
403	Agent-Upload deaktiviert (kein Token gesetzt); beim Android-Master zusätzlich: Quelle nicht im lokalen Netzwerk

6.5 Anforderungen an das Netzwerk

- Master und Agenten befinden sich im selben Netz; der Master ist über seine LAN-IP erreichbar.
- Windows-Master (Variante B): eingehende Regel für TCP 8765 nur im privaten Netzwerkprofil, nicht am Router/Internet freigeben.
- Android-Master (Variante C): Android 17 blockiert lokalen Netzwerkverkehr ohne die Berechtigung ACCESS_LOCAL_NETWORK; sie wird beim ersten Start abgefragt.
- Empfohlen: feste IP bzw. DHCP-Reservierung für den Master, damit die Master-URL der Agenten stabil bleibt.
- Internetzugang nur für den CVE-Abgleich (HTTPS zur NVD) und, unter Windows, für die Installation der Python-Pakete.

7 Dashboard-Server: Referenz

7.1 Endpunkte

Methode / Pfad	Zweck	Hinweise
GET /	Liefert die Oberfläche	Trägt die lokale Server-IP ein
GET /static/*	CSS und JavaScript	–
GET /api/summary	Kennzahlen, Hosts, Zeitpunkt der letzten Aktualisierung	host_count, online_count, software_count, process_count, hosts, last_updated
POST /api/agent/report	Agent-Upload	Token-Prüfung; siehe Abschnitt 6
POST /api/ping	Ping einer einzelnen IP	Nur private, Loopback, Link-Local; sonst 400
POST /api/cve	CVE/CVSS-Abgleich für eine Softwareliste	Maximal 100 Einträge, sonst 400
GET /health	Statuscheck	Antwort: ok, bind

7.2 Datenhaltung

Die Inventardaten liegen in einer JSON-Datei (Windows: `data/inventory.json`; Android: interner App-Speicher). Das Schreiben erfolgt über eine Zwischendatei mit anschließendem Ersetzen. Struktur: `{ "updated_at": ..., "hosts": [ ... ] }`.

7.3 CVE/CVSS-Abgleich

- Je Software wird aus Name und Version ein Suchbegriff gebildet (Leerzeichen normalisiert, maximal 120 Zeichen).
- Abfrage der NVD CVE API 2.0 mit keywordSearch und resultsPerPage=8, Zeitlimit 8 Sekunden je Anfrage.
- Der CVSS-Wert stammt aus dem ersten verfügbaren Metrikblock in der Reihenfolge v4.0, v3.1, v3.0, v2; Ausgabe von Score und Schweregrad.
- Beschreibung: englischer Text, auf 350 Zeichen gekürzt; Link auf die NVD-Detailseite.
- Ohne Treffer, bei Fehlern oder bei nicht erreichbarer NVD bleibt die Trefferliste leer.

Aussagekraft des Abgleichs
Ein Schlüsselworttreffer ist keine bestätigte Betroffenheit. Versionsbereiche, CPE-Angaben, Herstellerhinweise und installierte Patches müssen separat geprüft werden. CVSS beschreibt die technische Schwere, nicht das unternehmensspezifische Risiko.

8 Android-App (Version 2.0.0)

Die App ist ein vollständiger Master auf dem Handy: Sie enthält einen eingebetteten HTTP-Server, der die Endpunkte des Windows-Dashboards nachbildet, und zeigt die unveränderte Weboberfläche in einem WebView an. Die Windows-Agenten melden sich direkt bei der App.

8.1 Komponenten

Komponente	Aufgabe
MainActivity	Kopfzeile „GOLIATH / CYBER DASHBOARD“, Statuszeile mit Agenten-URL, WebView auf http://127.0.0.1:8765/, Dialog „Server“ (URL und Token anzeigen/kopieren/speichern/neu erzeugen), Berechtigungsabfrage, Zurück-Verhalten.
DashboardServer	Eigener HTTP-Server auf Port 8765 mit allen Endpunkten, Token-Prüfung (konstante Zeit), Ping, NVD-Abfrage, JSON-Speicher.
ServerService	Vordergrunddienst (Typ specialUse) mit Benachrichtigung und Beenden-Aktion; hält WakeLock und WLAN-Lock.
assets/	Unveränderte Dateien index.html, style.css, app.js des Dashboards.

8.2 Erreichbarkeit der Endpunkte

Endpunkt	Aus dem WLAN/LAN	Aus der App (127.0.0.1)
POST /api/agent/report	Ja (Token, private Quell-IP)	Ja
GET /health	Ja	Ja
GET /, /static/*, /api/summary, /api/ping, /api/cve	Nein (403)	Ja

8.3 Berechtigungen

Berechtigung	Zweck
INTERNET, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE	Server, NVD-Abfrage, Netzwerkstatus
ACCESS_LOCAL_NETWORK (Laufzeit, Android 17)	Eingehende und ausgehende Verbindungen im lokalen Netzwerk
FOREGROUND_SERVICE, FOREGROUND_SERVICE_SPECIAL_USE	Dauerhafter Serverbetrieb im Vordergrunddienst
POST_NOTIFICATIONS (Laufzeit)	Anzeige der Dienst-Benachrichtigung
WAKE_LOCK	CPU und WLAN bleiben für den Serverbetrieb aktiv

8.4 Android-17-Anpassungen

- compileSdk und targetSdk 37, minSdk 26 (Android 8.0).
- Seitenränder und Tastatur werden über Window-Insets berücksichtigt (Edge-to-Edge).
- Zurück-Geste über OnBackInvokedDispatcher; auf der Startseite läuft der Server im Hintergrund weiter.
- Lokaler Netzwerkzugriff über die Laufzeitberechtigung ACCESS_LOCAL_NETWORK.

8.5 Unterschiede zum Windows-Dashboard

- Das Dashboard ist von außen nicht abrufbar, nur der Agent-Upload ist im LAN offen.
- Der Ping nutzt InetAddress.isReachable; Geräte, die Ping blockieren, erscheinen als „keine Antwort“.
- Das Handy selbst meldet sich nicht als Agent; Software- und Prozessinventar von Android wird nicht erfasst.

8.6 Projektstatus der App

Stand der Umsetzung
Die App liegt als Android-Studio-Projekt vor. Eine signierte oder installierbare APK wurde in der Erstellungsumgebung nicht gebaut (kein Android SDK), die App wurde daher nicht auf einem Gerät getestet. Der Build erfolgt mit Android Studio oder über den mitgelieferten GitHub-Actions-Workflow.

9 Technische Anforderungen

Komponente	Anforderung
Windows-Master	Windows 10 oder 11; Python 3.11 oder neuer (Launcher „py“ im PATH); Pakete Flask (3.x) und requests (2.31 bis unter 3.0), die der Starter installiert; Internet für Pakete und NVD.
Windows-Agent	Windows 10/11 mit PowerShell (enthalten); Leserechte auf die Uninstall-Registry-Schlüssel; Netzwerkzugang zum Master.
Android-Master	Android 8.0 oder neuer; ausgelegt für Android 17 (API 37); WLAN im selben Netz wie die Agenten; Berechtigungen aus Abschnitt 8.3.
Build der App	Android Studio (aktuell) mit Android-SDK-Plattform 37 und Build-Tools 37, JDK 17, Gradle-Plugin 8.13.0 (ggf. per Upgrade-Assistent anpassen); alternativ GitHub Actions.
Netzwerk	Gemeinsames LAN/WLAN, TCP-Port 8765 zum Master, ausgehend HTTPS (443) zur NVD.

10 Installation und Inbetriebnahme

10.1 Windows-Master und Agent

- ZIP in einen beschreibbaren Ordner entpacken, z. B. C:\Tools\esp32_s3_cyber_dashboard.
- START_DASHBOARD.bat ausführen: erstellt die virtuelle Umgebung, installiert die Pakete und fragt den Agent-Token ab (mindestens 32 Buchstaben/Ziffern, keine Sonderzeichen).
- Das Serverfenster geöffnet lassen; die Oberfläche läuft unter http://127.0.0.1:8765.
- agent\COLLECT_AND_SEND.bat starten, Dashboard-URL und denselben Token eingeben.
- Für Agenten auf anderen PCs den Server mit GOLIATH_BIND=0.0.0.0 starten und die Firewall nur für das private Netz öffnen.

10.2 Android-Master

- APK bauen (Android Studio: Build > Build APK(s) oder GitHub-Actions-Workflow) und auf dem Handy installieren.
- App starten, Berechtigungen erlauben (Benachrichtigungen, lokales Netzwerk).
- Auf „Server“ tippen: Agenten-URL und Token ablesen oder kopieren.
- Auf dem Windows-PC agent\COLLECT_AND_SEND.bat starten und beides eingeben.
- Für zuverlässigen Empfang bei ausgeschaltetem Display den Akkumodus der App auf „Unbeschränkt“ stellen.

11 Sicherheit und Datenschutz

11.1 Umgesetzte Schutzmaßnahmen

- Agent-Upload nur mit Token; ohne gesetzten Token ist der Upload deaktiviert.
- Annahme nur von Berichten mit privater, Loopback- oder Link-Local-Agent-IP; Ping nur zu solchen Adressen.
- Ping und Berichtsvalidierung arbeiten mit IP-Literalen, es erfolgt keine Namensauflösung auf Benutzereingaben.
- Ausgabe in der Oberfläche wird HTML-maskiert; es gibt keine Portscans, Exploits oder Remote-Befehle.
- Android: Dashboard und Lese-Endpunkte nur über Loopback; Token-Vergleich in konstanter Zeit; App-Backup deaktiviert.

11.2 Bekannte Risiken und Empfehlungen

Risiko	Empfehlung
Verkehr über HTTP unverschlüsselt; der Token schützt nicht die Vertraulichkeit	Nur in vertrauenswürdigen Netzen betreiben; für fremde Netze TLS ergänzen.
Windows-Master mit GOLIATH_BIND=0.0.0.0: Oberfläche und Lese-Endpunkte im LAN ohne Anmeldung erreichbar	Firewall auf das Heimnetz beschränken, Zugriff auf einzelne Rechner begrenzen, Zugangsschutz ergänzen.
Token wird im Klartext eingegeben und ist ein gemeinsames Geheimnis	Langen Zufallstoken verwenden, nicht wiederverwenden, bei Verdacht erneuern.
Android: Andere Apps auf dem Handy können den Loopback-Port 8765 ansprechen	Keine unbekannten Apps installieren; Zugriffsschutz für die Oberfläche bei Bedarf ergänzen.
Inventardaten enthalten Software- und Prozessinformationen	Zugriff auf Master-Gerät und Datei begrenzen; Datenschutzvorgaben und interne Freigaben beachten.

12 Bekannte Grenzen

- Der CVE-Abgleich ist eine heuristische Schlüsselwortsuche. Falsche und fehlende Treffer sind möglich.
- Die öffentliche NVD-API ohne Schlüssel begrenzt Anfragen; bei großen Inventaren fehlen deshalb Treffer.
- Agenten laufen nur manuell; es gibt keine automatische oder zeitgesteuerte Meldung.
- Der Online-Status gibt den Stand des letzten Berichts wieder, kein Live-Verhalten.
- Jeder Bericht ersetzt den vorherigen desselben host+ip; es gibt keine Historie.
- Maximal 100 gespeicherte Berichte, 2000 Software- und 2000 Prozesseinträge je Bericht, 1000 angezeigte Zeilen je Tabelle.
- Der Windows-Agent ist nur für Windows verfügbar; Android-Geräte liefern kein Inventar.
- Die Android-App wurde bisher nicht kompiliert und nicht auf einem Gerät getestet.

13 Anhang

13.1 Dateiübersicht

Paket	Datei	Beschreibung
Windows-Dashboard 1.1	START_DASHBOARD.bat	Installiert Abhängigkeiten, fragt Token ab, startet den Server
	app.py	Flask-Server mit allen Endpunkten
	templates/index.html, static/*	Weboberfläche (HTML, CSS, JavaScript)
	agent/COLLECT_AND_SEND.bat, collect_and_send.ps1	Windows-Agent
	data/inventory.json, requirements.txt	Inventarspeicher, Python-Abhängigkeiten
Android-Projekt 2.0.0	app/src/main/java/.../MainActivity.java	Oberfläche der App
	DashboardServer.java, ServerService.java	Eingebetteter Server, Hintergrunddienst
	app/src/main/assets/*	Dashboard-Oberfläche
	agent/*, .github/workflows/build-apk.yml	Windows-Agent, Cloud-Build der APK

13.2 Glossar

Begriff	Erklärung
Master	Zentrale Instanz (Dashboard-Server), die Berichte der Agenten empfängt und darstellt.
Agent	Programm auf dem Ziel-PC, das Inventardaten erfasst und an den Master sendet.
CVE	Common Vulnerabilities and Exposures: eindeutige Kennung einer bekannten Sicherheitslücke.
CVSS	Common Vulnerability Scoring System: Bewertung der technischen Schwere einer Sicherheitslücke (0 bis 10).
NVD	National Vulnerability Database des NIST, Quelle der CVE- und CVSS-Daten.
Loopback	Adresse 127.0.0.1, erreichbar nur vom eigenen Gerät.

13.3 Versionshistorie

Version	Datum	Änderung
Dashboard 1.1	11.10.2026	Korrigierte Windows-Software-Inventarisierung, deutsche README, Token-Abfrage beim Serverstart
Android 1.0.0	13.10.2026	WebView-Client mit Server-Eingabe und Android-Agent-Bericht
Android 2.0.0	13.10.2026	Eingebetteter Master-Server, Android 17 (API 37), Vordergrunddienst, Token-Dialog
Dokumentation 1.0	13.10.2026	Erste Fassung dieser Projektdokumentation